

The Roofline Model, Worked Out by Hand

ONE CHART THAT TELLS YOU WHICH OPTIMIZATION WILL ACTUALLY HELP — A COMPLETE NUMERICAL WALK-THROUGH

What this document does. It builds the *roofline model* from first principles — the single most useful diagram for reasoning about any kernel on any accelerator — and then uses it to predict, on paper, exactly how much of an NVIDIA H100 our 70-billion-parameter *Nexus* model can extract in each phase of inference. Nothing is asserted without arithmetic. Every number is produced by a small reference script (Appendix A, `m01_t02_engine.py`) so the figures are exact and reproducible. By the end you will be able to plot decode and prefill on the roofline, read off their percent-of-peak, and name the one optimization the chart prescribes for each.

The one idea. A chip has two ceilings: a flat *compute roof* (F_{peak}) and a sloped *memory roof* ($\text{AI} \times B_{\text{HBM}}$). Where a workload lands under those roofs — decided entirely by its *arithmetic intensity* — tells you whether buying FLOPs or saving bytes is the lever that moves it. We make that claim quantitative.

CONCEPTS COVERED, IN ORDER

arithmetic intensity $\text{AI} = \text{FLOPs}/\text{bytes}$; the compute roof and the memory roof; achievable $= \min(\text{AI} \times B_{\text{HBM}}, F_{\text{peak}})$; the ridge point $F_{\text{peak}}/B_{\text{HBM}}$; the H100 ridge $= 295.5 \text{ FLOP/B}$; ridge points across the 2026 accelerator landscape; decode ($\text{AI} \sim 1$) and prefill ($\text{AI} \sim 6200$) placed on the chart; what the roofline prescribes in each regime; and how batching walks decode rightward.

INTERNAL LEARNING MATERIAL. NUMBERS ARE REPRODUCIBLE FROM APPENDIX A. v1.0

Contents

1	The system, and the question the roofline answers	2
2	Step 1 — Arithmetic intensity: the x-axis	2
3	Step 2 — The two roofs: the y-axis ceilings	3
4	Step 3 — The ridge point: where the roofs cross	3
5	Step 4 — The roofline, drawn	4
6	Step 5 — Reading off the percent-of-peak	4
7	Step 6 — The ridge point across the 2026 landscape	5
8	Step 7 — What the chart prescribes (and how batching moves you)	5

9 The argument, at a glance	7
9.1 Equation cheat-sheet	7
Appendix A — Reference implementation	7

1 The system, and the question the roofline answers

We reuse the concrete instance from Topic 1 so every quantity is a number you can check.

Quantity	Symbol	Value here
Parameters	P	70×10^9
Precision	—	FP16 (2 bytes/param)
Hidden width	d_{model}	8192
Decoder layers	N_{layers}	80
HBM bandwidth (H100 SXM, HBM3)	B_{HBM}	$3.35 \text{ TB/s} = 3.35 \times 10^{12} \text{ B/s}$
FP16 tensor-core peak (H100)	F_{peak}	$990 \text{ TFLOP/s} = 9.9 \times 10^{14} \text{ FLOP/s}$
Prompt length	T_{prompt}	6200 tokens
Output length	N_{out}	180 tokens

We use decimal units throughout: $1 \text{ GB} = 10^9 \text{ B}$, $1 \text{ TB/s} = 10^{12} \text{ B/s}$.

Intuition

A GPU is two machines bolted together: a stack of arithmetic units that can do F_{peak} FLOP/s, and a memory bus that can deliver B_{HBM} bytes/s. For any given kernel, *one* of those two is the binding constraint and the other has slack. The roofline is just a picture of which one binds — and therefore which one is worth spending money on. Optimizing the slack resource is wasted effort; the roofline keeps you honest about which is which.

2 Step 1 — Arithmetic intensity: the x-axis

The roofline plots achievable performance against **arithmetic intensity** (AI): how many FLOPs of useful work you get per byte of memory traffic.

$$\text{AI} = \frac{\text{FLOPs of work}}{\text{bytes moved to/from memory}} \quad [\text{FLOP/B}].$$

Take one weight matrix $W \in \mathbb{R}^{d \times d}$ with $d = d_{\text{model}} = 8192$. Reading it once costs $d^2 \cdot 2$ bytes; a matmul of an $[n \times d]$ activation against it costs $2nd^2$ FLOPs.

The same matrix, two regimes

$$\text{bytes} = d^2 \cdot 2 = 8192^2 \cdot 2 = 1.342 \times 10^8 \text{ B} = 134 \text{ MB}.$$

	rows n	FLOPs = $2nd^2$	AI = FLOPs/134 MB
decode	1	1.342×10^8	1.0 FLOP/B
prefill	6200	8.321×10^{11}	6200 FLOP/B

The bytes are *identical* in both rows — you read W once either way. Only the work done with those bytes differs. Decode does 1 FLOP per byte; prefill does 6200.

AI is a property of the *workload*, not the chip

Arithmetic intensity is decided by the shape of your matmul — specifically the batch (row) dimension n . It says nothing about hardware. The hardware enters in Step 2, as the two roofs you measure AI against. Separating "what the workload asks for" (AI) from "what the chip can give" (the roofs) is the whole conceptual trick of the roofline.

3 Step 2 — The two roofs: the y-axis ceilings

At a given AI, what is the most performance the hardware can deliver? Two limits apply simultaneously, and you get the smaller of them.

The two roofs and the achievable performance

$$\begin{aligned} \text{compute roof (flat): } P_{\text{compute}} &= F_{\text{peak}} \\ \text{memory roof (sloped): } P_{\text{memory}} &= \text{AI} \times B_{\text{HBM}} \\ \text{achievable: } P &= \min(\text{AI} \times B_{\text{HBM}}, F_{\text{peak}}). \end{aligned}$$

Why the memory roof is $\text{AI} \times B_{\text{HBM}}$

If the bus delivers B_{HBM} bytes per second, and each byte feeds AI FLOPs of work, then the work rate the bus can sustain is

$$\underbrace{B_{\text{HBM}}}_{\text{B/s}} \times \underbrace{\text{AI}}_{\text{FLOP/B}} = \text{AI} \times B_{\text{HBM}} \quad [\text{FLOP/s}].$$

This is a straight line through the origin on log-log axes: double the AI, double the deliverable FLOP/s — *until* you hit the flat compute roof and the chip simply cannot go faster no matter how much arithmetic each byte carries.

Intuition

Think of B_{HBM} as a conveyor belt delivering ingredients (bytes) and F_{peak} as the kitchen's maximum cooking rate (FLOPs). If each ingredient makes many dishes (high AI), the kitchen is the bottleneck — you are *compute-bound*. If each ingredient makes one dish (low AI), the belt is the bottleneck and the kitchen sits idle — you are *memory-bound*. The roofline is the graph of "fastest you can serve dishes" as the dishes-per-ingredient ratio slides from low to high.

4 Step 3 — The ridge point: where the roofs cross

The two roofs meet at one special AI — the **ridge point**. To its left you are memory-bound; to its right, compute-bound. Set the roofs equal:

$$\text{AI} \times B_{\text{HBM}} = F_{\text{peak}} \implies \text{ridge} = \frac{F_{\text{peak}}}{B_{\text{HBM}}}.$$

H100 ridge point

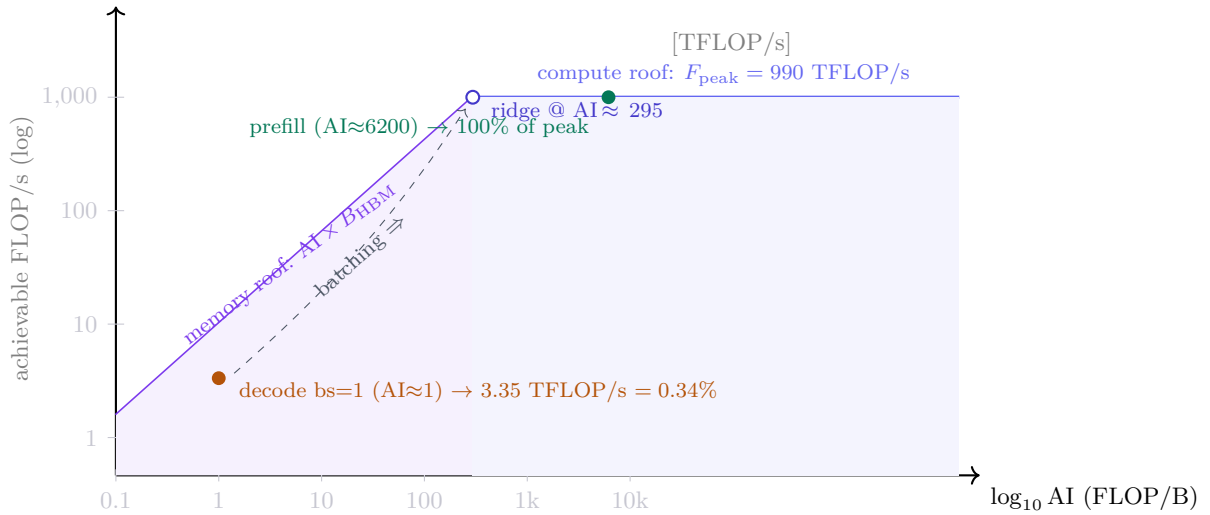
$$\text{ridge}_{\text{H100}} = \frac{F_{\text{peak}}}{B_{\text{HBM}}} = \frac{990 \times 10^{12}}{3.35 \times 10^{12}} = \frac{9.90 \times 10^{14}}{3.35 \times 10^{12}} = 295.5 \text{ FLOP/B. } \checkmark$$

The number worth memorizing

The H100 needs **295.5 FLOPs of work per byte** of HBM traffic before its tensor cores are fully fed. Below that, it is starved; above it, saturated. Every accelerator has its own ridge — it is the chip's "appetite" for arithmetic per byte. Memorize 295 for the H100 and you can classify any kernel in your head.

5 Step 4 — The roofline, drawn

Here is the chart for the H100, with our two workloads placed on it. The x-axis (AI) and y-axis (achievable FLOP/s) are both log scale.

**Watch out**

The sketch is schematic — the axes are log scale, so the memory roof is a straight line of slope 1, not the curve your eye might expect. What matters is the *topology*: decode sits far down-left on the sloped roof (deeply memory-bound), prefill sits up-right on the flat roof (compute-saturated), and the ridge at $\text{AI} \approx 295$ is the watershed between them.

6 Step 5 — Reading off the percent-of-peak

Now plug AI into $P = \min(\text{AI} \times B_{\text{HBM}}, F_{\text{peak}})$ and divide by F_{peak} to get utilization.

Decode and prefill, exactly

decode (AI = 1) : $P = \min(1 \times 3.35 \times 10^{12}, 9.9 \times 10^{14}) = 3.35 \times 10^{12} \text{ FLOP/s}$,

$$\frac{P}{F_{\text{peak}}} = \frac{3.35 \times 10^{12}}{9.9 \times 10^{14}} = \mathbf{0.34\%}.$$

prefill (AI = 6200) : $P = \min(6200 \times 3.35 \times 10^{12}, 9.9 \times 10^{14}) = 9.9 \times 10^{14} \text{ FLOP/s}$,

$$\frac{P}{F_{\text{peak}}} = \mathbf{100\%}. \quad \checkmark$$

Regime	AI (FLOP/B)	vs. ridge 295.5	Achievable
Decode	1	$\ll$ ridge $\Rightarrow$ memory-bound	3.35 TFLOP/s = 0.34% of peak
Prefill	6200	$\gg$ ridge $\Rightarrow$ compute-bound	990 TFLOP/s = 100% of peak

The gap between 1 and 295 is why this field exists

Decode asks for 1 FLOP/B; the H100 wants 295.5. That $295\times$ shortfall means the tensor cores run at 0.34% of rating during decode — not slow, *starved*. The entire job of LLM serving is closing that gap: moving decode rightward toward the ridge.

7 Step 6 — The ridge point across the 2026 landscape

The ridge is just $F_{\text{peak}}/B_{\text{HBM}}$, so every accelerator has its own. A lower ridge means a chip is fed sooner — better for memory-bound decode. (FP16 peaks.)

Accelerator	BW (TB/s)	FP16 (TFLOP/s)	ridge (FLOP/B)	Note
NVIDIA A100 SXM 80GB	2.04	312	152.9	legacy fleet
NVIDIA H100 SXM	3.35	990	295.5	our baseline
NVIDIA H200 SXM	4.80	990	206.2	same cores, +BW
NVIDIA B200	8.00	2250	281.2	frontier
AMD MI300X	5.30	1307	246.6	memory-heavy
AMD MI325X	6.00	1307	217.8	long-context KV
AWS Trainium2	2.90	650	224.1	cost-optimized
Google TPU v5p	2.76	459	166.3	dense matmul

Mini-example — the counterintuitive H200 story

The H200 has the *same* tensor cores as the H100 — identical 990 TFLOP/s peak — but 4.80 TB/s of HBM3e instead of 3.35 TB/s. Its ridge drops from 295.5 to 206.2 FLOP/B: the chip is fed with 30% less arithmetic per byte. For decode-dominated Nexus, that lower ridge plus 76% more capacity is a $\sim 43\%$ throughput win *with zero code changes*. For inference, bandwidth beats FLOPs — the ridge-point arithmetic proves it.

8 Step 7 — What the chart prescribes (and how batching moves you)

The roofline does not just diagnose — it prescribes. *Your position relative to the ridge tells you which optimization has leverage.*

Regime	Where you are	What the chart says to do
Decode	far left, on the sloped memory roof ($AI \ll \text{ridge}$)	Read fewer/shared bytes. Quantization shrinks bytes; <i>batching</i> raises AI by reusing each weight read across many requests. Buying FLOPs does nothing.
Prefill	far right, on the flat compute roof ($AI \gg \text{ridge}$)	Do less math. FlashAttention, FP8 tensor cores, 2:4 sparsity, kernel fusion. More bandwidth does nothing here.

The highest-leverage decode move is **batching**: one weight read serves all b requests in the batch, so decode AI rises linearly with the batch size,

$$AI(b) = \frac{2b}{\text{bytes per param}}.$$

Batching walks decode rightward toward the ridge

With 2 bytes/param, $AI(b) = 2b/2 = b$. Apply $P = \min(AI \times B_{\text{HBM}}, F_{\text{peak}})$:

batch b	AI	achievable	% of peak
1	1.0	3.35 TFLOP/s	0.34%
16	16	53.6 TFLOP/s	5.4%
64	64	214 TFLOP/s	21.7%
128	128	429 TFLOP/s	43.3%
296	296	990 TFLOP/s	100% (ridge)

Why every later module obsesses over batching

From batch 1 to batch 128, compute utilization climbs $125\times$ ($0.34\% \rightarrow 43\%$) without a single new FLOP of hardware — purely by reusing each weight read. The roofline predicted exactly this: batching slides decode rightward along the memory roof until, around batch 296, it finally touches the ridge and becomes compute-bound. That is why continuous batching (Module 05) is the highest-leverage technique in all of LLM serving.

Watch out

Batching raises *throughput per GPU*, not single-stream *latency*: each user still waits one full weight-read per token. And you cannot batch past your KV-cache memory budget — which is why Modules 03–04 (attention and KV cache) exist to make those large batches fit. The roofline tells you *where* the win is; the rest of the course is about *affording* it.

9 The argument, at a glance

#	Step	Result
1	x-axis	AI = FLOPs/bytes; decode = 1, prefill = 6200
2	roofs	achievable = $\min(\text{AI} \times B_{\text{HBM}}, F_{\text{peak}})$
3	ridge	$F_{\text{peak}}/B_{\text{HBM}} = 295.5 \text{ FLOP/B}$ ✓
4	chart	decode bottom-left (memory), prefill top-right (compute)
5	%peak	decode 0.34%, prefill 100%
6	landscape	H200 ridge 206, MI300X 247, B200 281
7	prescription	decode → fewer/shared bytes; prefill → less math

9.1 Equation cheat-sheet

Arithmetic intensity	AI = FLOPs/bytes moved
Compute roof	$P_{\text{compute}} = F_{\text{peak}}$
Memory roof	$P_{\text{memory}} = \text{AI} \times B_{\text{HBM}}$
Achievable FLOP/s	$P = \min(\text{AI} \times B_{\text{HBM}}, F_{\text{peak}})$
Ridge point	$\text{ridge} = F_{\text{peak}}/B_{\text{HBM}}$
Bound test	AI < ridge ⇒ memory-bound
Decode AI vs. batch	$\text{AI}(b) = 2b/(\text{bytes per param})$

Appendix A — Reference implementation

Every number above is produced by `m01_t02_engine.py` (provided alongside this PDF). Run it to reproduce all figures; change B_{HBM} or F_{peak} to compute the ridge point and percent-of-peak for any other accelerator.

```

1  """
2  Reference implementation for: Module 01 - Topic 2
3  "The Roofline Model - one chart that predicts performance."
4
5  Every number printed here is transcribed (rounded) into the worked-by-hand PDF,
6  so the arithmetic in the document is exact and self-consistent.
7
8  Convention: 1 GB = 1e9 bytes, 1 TB/s = 1e12 B/s (decimal, the convention
9  vendors use for HBM bandwidth and model "size in GB").
10 """
11
12 # -----
13 # 0. The system under study: one 70B decoder on one H100 SXM.
14 # -----
15 P          = 70e9          # parameters (Llama-3-70B class)
16 bytes_fp16 = 2            # bytes per parameter at FP16
17 d_model    = 8192         # hidden width
18 n_layers   = 80
19 B_hbm      = 3.35e12      # H100 SXM HBM3 peak bandwidth, bytes/s
20 F_peak     = 990e12       # H100 FP16 tensor-core peak, FLOP/s
21 T_prompt   = 6200        # Nexus prompt length (tokens)
22 N_out      = 180         # Nexus output length (tokens)
23
24 # -----
25 # 1. Arithmetic intensity: FLOPs of useful work / bytes of memory traffic.
26 #   One weight matrix W in R{d x d}; read once = d*d*2 bytes.
27 #   matmul of an [n x d] activation by [d x d] costs 2*n*d*d FLOPs.

```

```

28 # -----
29 def matmul_flops(n, d): return 2 * n * d * d
30 def weight_bytes(d): return d * d * bytes_fp16
31
32 w_b = weight_bytes(d_model)
33 print("== 1. Arithmetic intensity of one d x d projection ==")
34 print(f"weights read once = d^2 * 2B = {w_b:.3e} B = {w_b/1e6:.0f} MB")
35 for name, n in [("decode (1 token)", 1), ("prefill (6200 tokens)", T_prompt)]:
36     flops = matmul_flops(n, d_model)
37     ai = flops / w_b
38     print(f" {name:24s}: FLOPs={flops:.3e} AI={ai:8.2f} FLOP/B")
39
40 ai_decode = matmul_flops(1, d_model) / w_b
41 ai_prefill = matmul_flops(T_prompt, d_model) / w_b
42
43 # -----
44 # 2. The two roofs and the achievable performance.
45 # compute roof = F_peak (flat)
46 # memory roof = AI * B_hbm (sloped, slope = bandwidth)
47 # achievable = min(AI * B_hbm, F_peak)
48 # -----
49 def achievable(ai, bw=B_hbm, fp=F_peak): return min(ai * bw, fp)
50 print("\n== 2. Achievable FLOP/s from the roofline ==")
51 for name, ai in [("decode", ai_decode), ("prefill", ai_prefill)]:
52     ach = achievable(ai)
53     print(f" {name:8s}: min({ai:.2f}*{B_hbm:.2e}, {F_peak:.2e}) = {ach:.3e} FLOP/s "
54           f"({100*ach/F_peak:.2f}% of peak)")
55
56 # -----
57 # 3. The ridge point: AI where the two roofs cross.
58 # -----
59 ridge = F_peak / B_hbm
60 print("\n== 3. Ridge point of the H100 ==")
61 print(f"ridge = F_peak / B_hbm = {F_peak:.2e} / {B_hbm:.2e} = {ridge:.1f} FLOP/B")
62 print(f"decode AI={ai_decode:.2f} vs ridge {ridge:.0f} -> {'COMPUTE' if ai_decode>ridge else 'MEMORY'}-bound")
63 print(f"prefill AI={ai_prefill:.0f} vs ridge {ridge:.0f} -> {'COMPUTE' if ai_prefill>ridge else 'MEMORY'}-bound")
64
65 # -----
66 # 4. Ridge points across the 2026 accelerator landscape (section 06).
67 # ridge = peak FP16 FLOP/s / HBM bandwidth.
68 # -----
69 accel = [
70     # name, BW TB/s, FP16 TFLOP/s
71     ("A100 SXM 80GB", 2.04, 312),
72     ("H100 SXM", 3.35, 990),
73     ("H200 SXM", 4.80, 990),
74     ("B200", 8.00, 2250),
75     ("MI300X", 5.30, 1307),
76     ("MI325X", 6.00, 1307),
77     ("Trainium2", 2.90, 650),
78     ("TPU v5p", 2.76, 459),
79 ]
80 print("\n== 4. Ridge points across accelerators (ridge = F_peak / BW) ==")
81 print(f" {'GPU':16s} {'BW TB/s':>8s} {'FP16 TFLOP/s':>13s} {'ridge FLOP/B':>13s}")
82 for name, bw, fp in accel:
83     r = (fp * 1e12) / (bw * 1e12)
84     print(f" {name:16s} {bw:8.2f} {fp:13.0f} {r:13.1f}")
85
86 # -----
87 # 5. Place decode and prefill on the roofline; predict % of peak.
88 # -----
89 print("\n== 5. Where Nexus lives on the H100 roofline ==")
90 for name, ai in [("decode (AI~1)", ai_decode), ("prefill (AI~6200)", ai_prefill)]:

```



```

91     ach = achievable(ai)
92     regime = "MEMORY-bound" if ai < ridge else "COMPUTE-bound"
93     print(f"   {name:18s}: {regime:13s} achievable={ach/1e12:8.2f} TFLOP/s "
94           f"   f"= {100*ach/F_peak:6.2f}% of peak")
95
96 # -----
97 # 6. Batching walks decode rightward: AI(decode, batch b) = 2b / D_bytes.
98 #   Each weight byte is reused by b requests -> AI scales linearly with b.
99 # -----
100 print("\n== 6. Batching moves decode along the roofline (AI = 2b / bytes_per_param) ==")
101 for b in [1, 16, 64, 128, 296]:
102     ai_b = (2 * b) / bytes_fp16          # FLOP per weight byte at batch b
103     ach = achievable(ai_b)
104     print(f"   batch={b:4d}: AI={ai_b:7.1f} FLOP/B   achievable={ach/1e12:8.2f} TFLOP/s "
105           f"   f"= {100*ach/F_peak:6.2f}% of peak   ({'compute' if ai_b>=ridge else 'memory'}-bound)")
106
107 # -----
108 # 7. The optimization the chart predicts for each regime.
109 # -----
110 print("\n== 7. What the roofline prescribes ==")
111 print("   decode   (AI=1 << 295): read fewer/shared bytes -> quantization, batching")
112 print("   prefill  (AI>>295)      : do less math -> FlashAttention, FP8, sparsity")

```

— END OF DOCUMENT —